

# 032 实验指南

## 1 本次实验只比较一个固定训练问题

准备正文与answers.pdf。先修011、018、031；实际运行需要CPU PyTorch、NumPy，环境步骤见README。32行全部为合成训练数据，本讲没有独立测试集或真实任务排名。计数单位必须区分：优化器更新步数、参与更新的样本呈现次数、独立数据行数、实际墙钟时间。

本讲有三个层次：有限总体精确枚举解释抽样；一个可解标量模型解释噪声平台；一个13参数小网络比较固定预算。不要把不同层次的结论互相替代，例如用固定theta的方差公式直接宣称随机重排训练每一步都条件无偏。

## 2 从干净环境生成确定性结果

```
python experiment.py --output my-run
python -m unittest -v test_experiment
```

脚本先校验data/samples.csv、order.json、config.json，再完成全部计算与序列化，最后写结果。输入字节变化时拒绝固定报告。my-run会产生七份确定性文件，不生成计时文件；它不会把提供的测量秒数假装成当前运行秒数。

对比my-run与outputs中同名的七份数值结果。实验与Notebook均应从干净进程或清空内核开始运行；实际验证使用新进程InProcessKernel，没有测试浏览器Jupyter与socket通信。用户自己的新Anaconda安装和跨平台环境亦未实测。

## 3 用全部可能批次检查方差

打开sampling-exact.json。固定四个2维梯度，B=1、2、3、4分别枚举有放回有序结果与不放回子集。Fraction字段保存有理数精确结果，covariance字段只是对应浮点数便于画图。

手工列出B=2不放回的六个子集。第1坐标均值为2、1、0、0、-1、-2，因此方差为5/3；第2坐标与交叉项也必须算，不能只对上一个对角元素就断言整个协方差正确。与有放回的5/2及完全重复同一随机样本的5比较。

接着看network-sampling.json：它固定同一组13参数，只在前8行上枚举子集。检查B=8时方差为0。训练过程中theta会改变，这份文件的固定点协方差不能当成全部训练时刻的噪声估计。

## 4 故意破坏无偏前提

nonuniform字段给出均匀目标梯度0、非均匀直接估计期望-1，以及重要性修正后0。重新手算每一项的概率乘值，解释为什么四个修正值的普通算术均值不是要计算的期望。

reshuffle\_conditional\_counterexample记录两种首个目标。先读到+1后 $\theta=.5$ ，剩余目标-1给梯度1.5，全梯度却为.5。必须在给定历史后比较；把正反两种完整顺序再平均，会掩盖问题所要求的条件偏差。

这些例子用于定位理论前提，不说明不放回采样“不可用”。若代码使用shuffle=True，应记录随机重排，不应在报告里自动写成独立有放回SGD。

## 5 一次完整网络更新不能只看最后loss

打开first-batch-trace.json。使用ID25、9、6、4，核对每行X、A、H、预测、残差及loss。然后按正文第7节依次检查d\_prediction、d\_hidden、d\_affine、d\_input，以及13个参数梯度与更新后前向。

以下完整程序在本讲目录运行。它用独立标量循环参考核对损失与全部梯度，再对第0个参数做中心差分；完整测试还会覆盖所有13个参数。

```
import numpy as np
import experiment as e
from test_experiment import scalar_reference
x, y, orders, cfg = e.load_inputs()
theta = np.array(cfg['initial_parameters'], dtype=np.float64)
ids = orders[0][:4]
trace = e.first_batch_trace(x, y, orders, cfg)
loss, gradient = scalar_reference(theta, x[ids], y[ids])
print('同一批loss', loss, trace['loss'])
print('全部梯度最大差', np.max(np.abs(gradient-trace['gradient'])))
plus, minus = theta.copy(), theta.copy()
plus[0] += 1e-5
minus[0] -= 1e-5
fd = (scalar_reference(plus, x[ids], y[ids])[0]
      - scalar_reference(minus, x[ids], y[ids])[0]) / 2e-5
print('参数0差分与链式梯度', fd, gradient[0])
print('更新后同批loss', trace['next_loss'])
```

预期原损失约.0640381149，更新后约.0633289804。不要把这一小步的下降当作所有步必降；更不要把这一批损失直接当完整32行目标。

## 6 预算表与学习率比较

查看budget-comparison.csv和trajectories.json。为每条曲线补全四列：B、更新步数、样本呈现次数、学习率。equal\_examples用相同1280次呈现，equal\_steps用相同40步；linear\_scaled\_examples用相同1280次呈现，但学习率 $=.02 \times B/4$ 。

所有方案使用相同预先保存ID流前缀。检查used\_ids应等于该流的相应前缀，不仅检查长度。每次曲线纵轴都在同一完整32行数据上计算，避免不同小批损失的抽样噪声造成不可比。完整风险日志开销没有计入“训练样本呈现”，它在计时分支中也被关闭。

报告可以说：在这个固定数据、初值和学习率协议下出现某条轨迹。不能说：B=1普遍优于B=32，或线性缩放恒成立。三种预算协议本身已经展示了结论对选择的敏感性，没有进行公平的全面超参数搜索。

## 7 重新测量当前时间

```
python benchmark.py --output my-timing.json --repetitions 5
```

先读输出中的protocol。一次预热后，五次重复轮换配置顺序；计时包括完整train(record=False)调用的校验、初始化、索引、前后向、更新与保护，不包括读取数据、导入、每步全风险日志和写文件。最终状态与带日志分支逐项核对相同。

画时间时使用原始重复值与中位数，保留范围；它不是置信区间。报告CPU、线程、dtype、版本和测量窗口。即使数值输出可重复，运行秒数也会因共享CPU争用、调度与缓存状态波动。不要强制新时间与提供的measured-timing.json一致，也不要覆盖它再把原图称为新测量。

## 8 提交与失败案例

提交四项：全部抽样分布的协方差核对；一份给定历史的条件偏差解释；同一首批完整前后向检查；有三种预算定义和独立计时协议的对照报告。附一例学习率过大失效：标量递推中 $\eta=2$ 不收缩、 $\eta>2$ 不稳定，批量再大也不能消去这个收缩系数问题。

测试还应拒绝错误shape、非finite值、bool批量、无效索引、预算超过保存顺序长度和已修改固定输入。失败前不创建新结果或覆盖旧sentinel。单文件原子替换不是多个输出在断电情形下的整体事务；文件哈希只校验身份，不证明实验问题设计合理。